Ciências da Computação – 6º Semestre Noturno (NACCOMP6)

Entrega 02 – Programação e Controle de Dispositivos no Arduino

Sweet Spot

Arthur Paltrinieri (24026559)
Beatriz de Souza Santos Rio Branco (23025365)
Fabiano Henrique Chou (24025991)
Marcelo Henrique de Souza (23025665)

2026

RESUMO

O projeto Sweet Spot propõe uma plataforma de treinamento solo de precisão no tênis, utilizando uma matriz projetada para 100 LEDs que representa alvos luminosos e prevendo integração com visão computacional e aplicação web. Esta entrega aborda a programação e o controle de dispositivos com Arduino, com ênfase no acionamento de LEDs por registradores de deslocamento, no uso de estruturas de decisão e repetição e na leitura de uma entrada digital. As fotografias coloridas registram a montagem física, um Arduino Uno conectado e LEDs iluminados na superfície de treino. O código-fonte completo incluído ao final foi dimensionado para 100 canais lógicos mediante 13 registradores 74HC595 em cascata, com botão de teste como entrada digital. A correspondência da arquitetura proposta com a fiação real e a operação simultânea dos 100 LEDs ainda dependem de validação elétrica em bancada.

Palavras-chave: Arduino. LEDs. Registradores de deslocamento. Controle digital. Sweet Spot.

1 INTRODUÇÃO

A Sweet Spot combina recursos de Internet das Coisas (IoT), visão computacional e gamificação para criar uma experiência de treino de precisão no tênis. Na concepção do subsistema luminoso, 100 LEDs representam possíveis posições-alvo; pequenos grupos consecutivos se acendem conforme o nível de dificuldade. Bolas de isopor substituem as bolas de tênis nesta fase de prototipação. A proposta global prevê que duas webcams e um modelo YOLO identifiquem a trajetória da bola e subsidiem o cálculo da posição de impacto.

A presente entrega tem escopo específico: documentação do subsistema embarcado projetado para controlar 100 LEDs e de uma rotina demonstrativa de leitura digital. As funcionalidades de rastreamento, cálculo da precisão geométrica, API Node.js e banco MySQL pertencem à arquitetura mais ampla e não são implementadas pelo sketch apresentado.

2 OBJETIVOS

O objetivo geral é apresentar uma solução programável em Arduino dimensionada para endereçar 100 LEDs como atuadores de alvos e ler ao menos uma entrada digital, de acordo com o enunciado da Entrega 02.

São objetivos específicos: gerar alvos luminosos em diferentes posições de uma matriz de 100 LEDs; ajustar a extensão de cada alvo conforme a dificuldade; utilizar variáveis, condições e laços; aplicar leitura digital com tratamento de ruído de botão; organizar uma pontuação demonstrativa e documentar a montagem com fotografias coloridas e referência ao vídeo da equipe.

3 COMPONENTES E ARQUITETURA DO PROTÓTIPO

Elemento	Função na solução	Situação observável
Arduino Uno	Montagem e distribuição dos sinais; referência de 13 x 74HC595 em cascata	Placa identificável na fotografia 3
Matriz planejada de 100 LEDs	Representação visual das 100 posições possíveis de alvo	Parte dos LEDs aparece acesa na fotografia 2; total de 100

Elemento	Função na solução	Situação observável
		não aferido
Matriz de LEDs	Representação visual dos alvos	LEDs acesos na fotografia 2
Fiação e alimentação	Interligação elétrica	Cabos visíveis; pinagem e tensões não aferidas
Entrada digital de teste	Simulação do evento de acerto no código de exemplo	Botão externo não identificado nas fotografias

A arquitetura de referência prevê 100 LEDs individualmente endereçáveis por 13 registradores de deslocamento 74HC595 em cascata, com oito saídas por CI (104 saídas possíveis, quatro não utilizadas). Três pinos do Arduino controlam dados, clock e latch; um quarto pino recebe o botão de teste. As fotografias mostram uma montagem com múltiplos circuitos integrados, mas não permitem identificar com segurança todos os modelos, a pinagem nem confirmar a presença de 13 registradores. Assim, os 100 canais representam a capacidade prevista no código, e não uma comprovação de que a montagem fotografada contém todos eles.

4 LÓGICA DE FUNCIONAMENTO

4.1 Inicialização

A função setup() configura três saídas digitais para transmissão serial aos 13 registradores previstos e um pino de entrada com resistor pull-up interno, inicializa a comunicação serial em 9600 bit/s, desliga as 104 saídas disponíveis e apresenta o primeiro alvo dentre os 100 LEDs válidos.

4.2 Controle dos alvos luminosos

A quantidade de LEDs acesos simultaneamente corresponde à largura do alvo. A função gerarAlvo() sorteia um ponto inicial entre as posições válidas de um conjunto de 100 LEDs e preenche um vetor de 13 bytes. Cada posição é mapeada para seu respectivo bit por divisão e resto por oito; a função escreverLeds() transmite todos os bytes em cascata com shiftOut(). As quatro saídas excedentes permanecem desligadas.

4.3 Níveis de dificuldade

O programa aceita os comandos 1 (fácil, seis LEDs consecutivos), 2 (médio, quatro LEDs) e 3 (difícil, dois LEDs) pelo Monitor Serial. A posição do alvo pode variar ao longo de todos os 100 LEDs endereçados, e o grupo aceso se torna menor

conforme aumenta a dificuldade. As dimensões físicas e a distribuição espacial desses alvos dependem do painel efetivamente montado.

4.4 Leitura da entrada digital

O código demonstra a utilização de um botão conectado ao pino D2 e ao GND, com INPUT_PULLUP. O acionamento é detectado por transição estável para nível baixo (LOW), com debounce de 35 milissegundos. Esse botão é um recurso didático para o requisito de leitura de entrada e não representa, por si só, a medição de impacto ou a leitura das câmeras.

4.5 Pontuação e controle de tempo

O tempo inicial configurado é de 30 segundos. Cada evento válido no botão adiciona três segundos e gera pontuação ilustrativa de 100, 70 ou 40 pontos conforme o tempo decorrido desde o surgimento do alvo. O cálculo considera a rapidez do acionamento, não a precisão espacial do movimento da bola.

4.6 Encerramento

A função loop() atualiza o cronômetro sem utilizar delay() na rotina principal; quando o tempo termina, a rotina apagarLeds() transmite 13 bytes nulos para desligar todas as saídas. O comando r reinicia a demonstração e seleciona novo alvo no conjunto de 100 posições.

5 CONEXÕES DO CÓDIGO DE REFERÊNCIA

Pino ou terminal	Ligação proposta
Arduino D8	DS/SER do primeiro entre 13 registradores 74HC595 previstos
Arduino D9	SH_CP/SRCLK comum aos 13 registradores
Arduino D10	ST_CP/RCLK comum aos 13 registradores
74HC595 Q7'	Q7' de cada CI ligado ao DS/SER do seguinte (12 enlaces)
Arduino D2	Botão de teste para GND (INPUT_PULLUP)
OE / MR / alimentação	OE em GND, MR em VCC; VCC e GND conforme o CI
Saídas dos registradores	100 LEDs nas posições 0 a 99,

Pino ou terminal	Ligação proposta
	cada um com limitação de corrente; quatro saídas livres (100 a 103)

As conexões apresentadas são propostas para uma arquitetura de 100 LEDs, não verificadas ponto a ponto nas fotografias. Antes de energizar o sistema, devem ser confirmados o modelo dos CIs, a continuidade do encadeamento Q7’–SER, a presença de resistores limitadores por LED, o desacoplamento, a corrente por saída e por CI e a capacidade da alimentação. A utilização de drivers externos ou transistores pode ser necessária conforme a corrente dos LEDs. É indispensável compartilhar uma referência de GND adequada com o Arduino. O programa, isoladamente, não comprova a segurança elétrica da montagem.

6 EVIDÊNCIAS FOTOGRÁFICAS

As três imagens coloridas a seguir foram fornecidas pela equipe e mostram etapas da montagem e do acionamento visual do protótipo. Elas registram os circuitos, o painel de alvos e o Arduino Uno. O link do vídeo de funcionamento disponibilizado pela equipe encontra-se logo abaixo das fotografias. Fotografias e link não substituem testes elétricos específicos da arquitetura prevista de 100 LEDs nem a conferência da entrada digital.

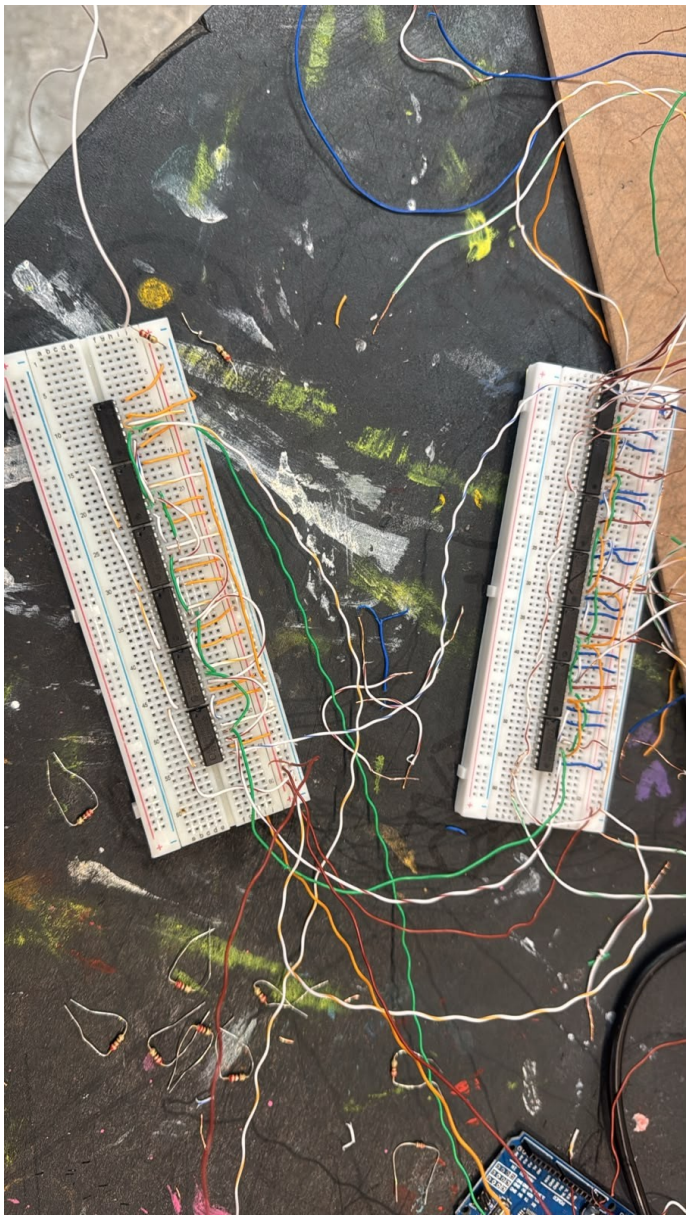


Figura 1 – Circuitos e conexões montados sobre protoboards.
Fonte: acervo da equipe Sweet Spot (2026).

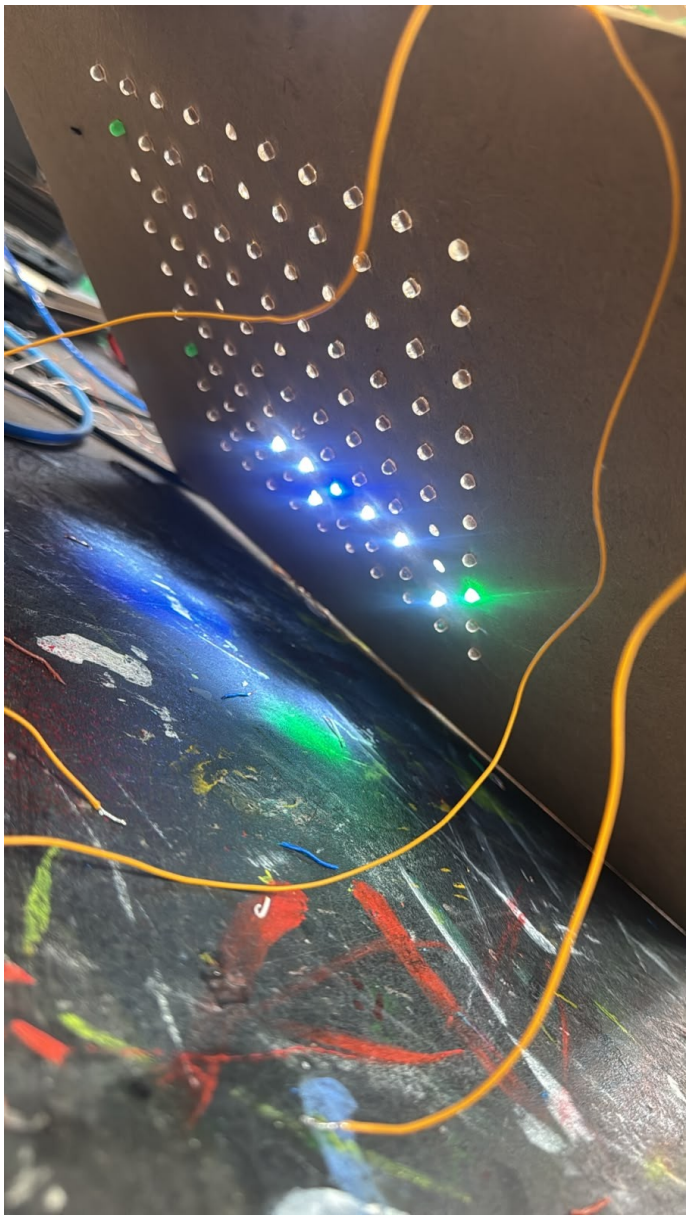


Figura 2 – LEDs acesos no painel de alvos da maquete.
Fonte: acervo da equipe Sweet Spot (2026).

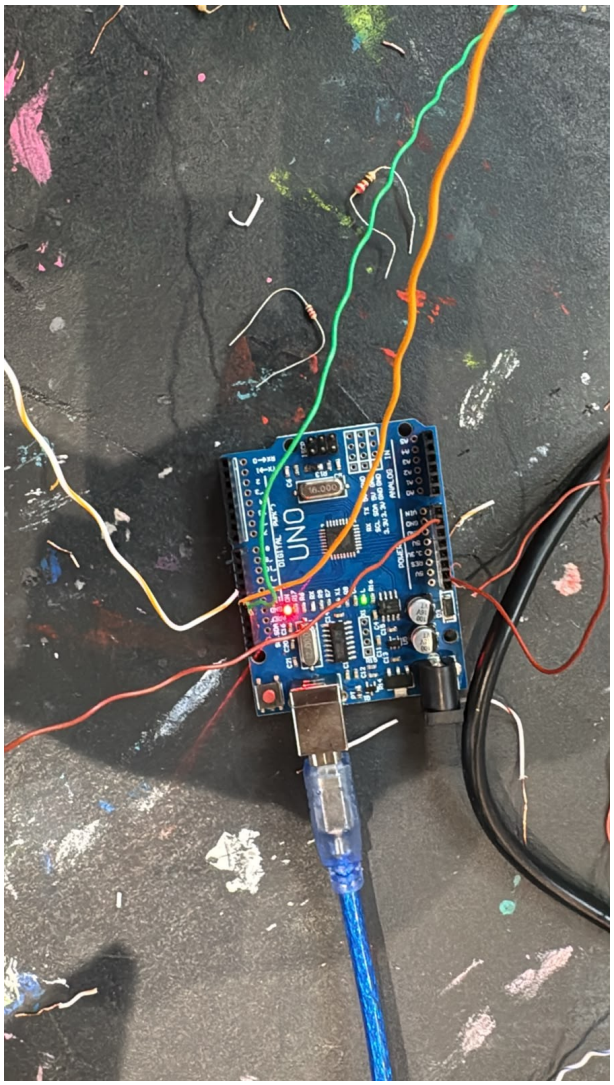


Figura 3 – Arduino Uno conectado à montagem física.
Fonte: acervo da equipe Sweet Spot (2026).

Vídeo do projeto em funcionamento (Google Drive):

https://drive.google.com/file/d/12p1CfveEmAp9gtoZEe40plwuV4mUutXS/view?usp=sharing

7 PROCEDIMENTO DE VALIDAÇÃO

Etapa	Procedimento	Evidência esperada
1	Conferir alimentação, resistores, corrente e cadeia de 13 registradores.	Nenhum CI ou fio aquece; GND e tensões verificados.
2	Carregar no Arduino o código para 100 LEDs.	Compilação, gravação e mensagens de 100 LEDs no Monitor Serial.
3	Enviar 1, 2 ou 3 no Monitor Serial.	Larguras de seis, quatro e dois LEDs em diferentes posições do painel.
4	Acionar o botão ligado a D2.	Evento e tempo de reação no Monitor Serial.
5	Aguardar o fim do cronômetro.	LEDs apagados e aviso de encerramento.
6	Enviar r.	Treino reiniciado com novo alvo.

O procedimento a seguir descreve como validar em bancada o código dimensionado para 100 LEDs. A equipe disponibilizou um vídeo do protótipo, indicado na seção de evidências; entretanto, sem inspeção técnica do vídeo, do sketch carregado e do Monitor Serial, os resultados detalhados deste roteiro não devem ser tratados como ensaios já certificados.

8 RELAÇÃO COM OS REQUISITOS DA ENTREGA

Requisito	Atendimento documentado
Código-fonte comentado	O Apêndice A contém código completo para 100 LEDs e 13 registradores 74HC595.
Descrição do funcionamento	Seções 3 a 5 descrevem a arquitetura de 100 canais, comandos e lógica.
Controle de atuador	LEDs acesos registrados na figura 2; controle de 100 canais definido

Requisito	Atendimento documentado
	em código.
Leitura de sensor/entrada	Implementada como botão digital no código demonstrativo; teste físico não comprovado.
Evidência prática	Três fotografias coloridas e link para o vídeo de funcionamento fornecido pela equipe.

9 CONCLUSÃO

A documentação apresenta uma proposta de controle luminoso escalável para 100 posições-alvo da Sweet Spot, preservando as evidências visuais coloridas da montagem física com Arduino Uno, protoboards e LEDs acesos. O programa do Apêndice A implementa a representação de 100 saídas por 13 registradores em cascata e inclui seleção de alvos, níveis de dificuldade, leitura digital e pontuação didática.

O registro audiovisual fornecido pela equipe está referenciado abaixo das fotografias. Para comprovar integralmente a operação de todas as 100 posições e o requisito de leitura de sensor, recomenda-se confrontar o vídeo com o programa efetivamente carregado, registrar o Monitor Serial, verificar eletricamente a cadeia de registradores e documentar o sensor utilizado. A integração com câmeras e a medição da precisão real da bola permanecem em escopo distinto.

REFERÊNCIAS

ARDUINO. Arduino Language Reference. Disponível em: https://docs.arduino.cc/language-reference/. Acesso em: 9 nov. 2026.

SWEET SPOT. Entrega 01 – Teoria da Computação e Linguagens Formais. Material da equipe, 2026.

SWEET SPOT. Registros fotográficos do protótipo. Acervo da equipe, 2026.

SWEET SPOT. Vídeo do protótipo em funcionamento. Acervo da equipe, 2026. Disponível em: https://drive.google.com/file/d/12p1CfveEmAp9gtoZEe40plwuV4mUutXS/view?usp=sharing

APÊNDICE A – CÓDIGO-FONTE COMENTADO DO ARDUINO

O código abaixo apresenta uma versão integral dimensionada para 100 LEDs lógicos em 13 registradores 74HC595 ligados em cascata. O vetor de 13 bytes representa 104 bits, dos quais quatro ficam desligados. As ligações indicadas são propostas e devem ser confrontadas com a montagem real. A entrada D2 continua sendo um botão didático: o sketch não detecta automaticamente o impacto da bola nem calcula precisão espacial.

```
/*
  Sweet Spot - Entrega 02: controle de 100 LEDs com Arduino Uno.
  Arquitetura de referencia: 13 x 74HC595 em cascata (104 saidas).
  LEDs 0 a 99 sao utilizados; as quatro saidas finais ficam em zero.
  Botao em D2 simula evento de acerto; nao mede impacto da bola.

  Ligacoes propostas (verificar na montagem real):
  D8  -> DS/SER do primeiro 74HC595
  D9  -> SH_CP/SRCLK de todos os registradores
  D10 -> ST_CP/RCLK de todos os registradores
  Q7' de cada 74HC595 -> DS/SER do proximo
  D2  -> botao para GND (INPUT_PULLUP)
  OE -> GND; MR -> VCC; GND comum e alimentacao adequada.
  Usar resistores para LEDs, desacoplamento e drivers quando
  necessarios. Respeitar os limites de corrente dos CIs e da fonte.
*/

const uint8_t PIN_DADOS = 8;
const uint8_t PIN_CLOCK = 9;
const uint8_t PIN_LATCH = 10;
const uint8_t PIN_SENSOR = 2;

const uint8_t TOTAL_LEDS = 100;
const uint8_t TOTAL_REGISTRADORES = (TOTAL_LEDS + 7) / 8; // 13
const unsigned long BONUS_MS = 3000UL;
const unsigned long DEBOUNCE_MS = 35UL;

uint8_t larguraAlvo = 4; // facil=6, medio=4, dificil=2
uint8_t inicioAlvo = 0;
unsigned long pontuacao = 0;
unsigned long acertos = 0;
unsigned long instanteAlvo = 0;
unsigned long tempoRestanteMs = 30000UL;
unsigned long ultimoLoop = 0;
unsigned long instanteMudanca = 0;
bool leituraAnterior = HIGH;
bool estadoEstavel = HIGH;
bool treinoAtivo = true;

// Envia 13 bytes: ultimo registrador primeiro, primeiro por ultimo.
// O byte zero controla os LEDs 0..7; o byte 12, os LEDs 96..99.
void escreverLeds(const uint8_t bits[TOTAL_REGISTRADORES]) {
  digitalWrite(PIN_LATCH, LOW);
  for (int8_t r = TOTAL_REGISTRADORES - 1; r >= 0; --r) {
    shiftOut(PIN_DADOS, PIN_CLOCK, MSBFIRST, bits[r]);
  }
  digitalWrite(PIN_LATCH, HIGH);
}

void apagarLeds() {
```

```
  uint8_t zeros[TOTAL_REGISTRADORES] = {0};
  escreverLeds(zeros);
}

// Seleciona um grupo contiguo dentro dos 100 LEDs logicos.
void gerarAlvo() {
  if (!treinoAtivo) return;
  inicioAlvo = (uint8_t)random(0, TOTAL_LEDS - larguraAlvo + 1);
  uint8_t bits[TOTAL_REGISTRADORES] = {0};
  for (uint8_t i = 0; i < larguraAlvo; ++i) {
    uint8_t led = inicioAlvo + i;
    bits[led / 8] |= (uint8_t)(1U << (led % 8));
  }
  escreverLeds(bits);
  instanteAlvo = millis();
  Serial.print(F("Novo alvo: LEDs "));
  Serial.print(inicioAlvo);
  Serial.print(F(" a "));
  Serial.println(inicioAlvo + larguraAlvo - 1);
}

void reiniciarTreino() {
  acertos = 0;
  pontuacao = 0;
  tempoRestanteMs = 30000UL;
  treinoAtivo = true;
  ultimoLoop = millis();
  gerarAlvo();
  Serial.println(F("Treino reiniciado (30 s)."));
}

// Pontuacao demonstrativa por rapidez, sem localizar a bola.
void registrarEventoSensor() {
  if (!treinoAtivo) return;
  unsigned long reacaoMs = millis() - instanteAlvo;
  unsigned int pontos = reacaoMs < 1000UL ? 100 :
                        reacaoMs < 2000UL ? 70 : 40;
  ++acertos;
  pontuacao += pontos;
  tempoRestanteMs += BONUS_MS;
  Serial.print(F("Evento do sensor | Reacao: "));
  Serial.print(reacaoMs);
  Serial.print(F(" ms | Pontos: "));
  Serial.print(pontos);
  Serial.print(F(" | Total: "));
  Serial.print(pontuacao);
  Serial.print(F(" | Acertos simulados: "));
  Serial.println(acertos);
  gerarAlvo();
}

void setup() {
  pinMode(PIN_DADOS, OUTPUT);
  pinMode(PIN_CLOCK, OUTPUT);
  pinMode(PIN_LATCH, OUTPUT);
  pinMode(PIN_SENSOR, INPUT_PULLUP);
  Serial.begin(9600);
  randomSeed(analogRead(A0)); // A0 livre: semente pseudoaleatoria
  apagarLeds();
  Serial.println(F("Sweet Spot | 100 LEDs | 1 facil, 2 medio, 3 dificil, r reiniciar"));
  reiniciarTreino();
}

void loop() {
```

```
  unsigned long agora = millis();
  unsigned long delta = agora - ultimoLoop;
  ultimoLoop = agora;

  if (treinoAtivo) {
    if (delta >= tempoRestanteMs) {
      tempoRestanteMs = 0;
      treinoAtivo = false;
      apagarLeds();
      Serial.println(F("Fim do treino. Pressione r para reiniciar."));
    } else {
      tempoRestanteMs -= delta;
    }
  }

  // Leitura digital com debounce e deteccao de pressionamento.
  bool leitura = digitalRead(PIN_SENSOR);
  if (leitura != leituraAnterior) instanteMudanca = agora;
  if ((agora - instanteMudanca >= DEBOUNCE_MS) &&
      (leitura != estadoEstavel)) {
    estadoEstavel = leitura;
    if (estadoEstavel == LOW) registrarEventoSensor();
  }
  leituraAnterior = leitura;

  if (Serial.available() > 0) {
    char comando = Serial.read();
    if (comando == 'r' || comando == 'R') {
      reiniciarTreino();
    } else if (comando >= '1' && comando <= '3') {
      if (comando == '1') larguraAlvo = 6;
      if (comando == '2') larguraAlvo = 4;
      if (comando == '3') larguraAlvo = 2;
      Serial.print(F("Dificuldade alterada. Largura: "));
      Serial.println(larguraAlvo);
      if (treinoAtivo) gerarAlvo();
    }
  }
}
```